R65 Performance Benchmark

6502 Assembler, R65 Pascal and Historical Context

This note records a small benchmark experiment undertaken while considering the historical feasibility of the MILL/TEKMILL artificial intelligence on original R65 hardware. The experiment compares an optimized 6502 Sieve of Eratosthenes with an equivalent R65 Pascal implementation on the current R65 emulator.

1. Historical 6502 Sieve

The starting point was the optimized historical 6502 Sieve of Eratosthenes. It uses an 8192-byte flag array, finds 1899 primes, and was reported at 0.74 seconds on the original 6502 system. The optimized version stops clearing composite entries once the prime exceeds 127, while continuing to count primes.

2. Conversion to the R65 assembler

The source was mechanically cleaned and then adapted to the R65 assembler. R65 symbols are limited to six letters/numbers, so names such as A.PNTR and GENERATE.PRIMES became APNTR and GENPRM. Apple-specific character-output and timing code was removed so that only the Sieve computation was measured.

Useful R65 assembler syntax rediscovered during the conversion:

```
LABEL EQU $1234 symbolic constant ORG $2000 program origin LDA =4 immediate constant LDA
=LABEL immediate high byte ASL A explicit accumulator mode ROL A NAME BSS 2 reserve two
bytes NAME WRD value initialized word END
```

Address	Use
$2000-$2080	Sieve program
$3500-$54FF	8192-byte flag array
$5500	End of flag array
$06/$07	APNTR
$08/$09	BPNTR
$1B/$1C	PRIME
$1D/$1E	COUNT
$1F	TIMES

The final source assembled with zero errors. It was invoked with GSB 2000. A one-run correctness test returned COUNT = $076B = 1899, confirming that the conversion was functioning correctly.

3. Measuring the current emulator

A single assembler Sieve completed too quickly for useful manual timing. The benchmark was therefore run in an inner loop of 255 executions and an outer loop of 40 calls, giving 10,200 Sieve executions.

> **Measured stopwatch time: 70.91 seconds for 10,200 executions.**

One assembler Sieve therefore takes 70.91 / 10,200 = **0.006952 s**, or approximately **6.95 ms**, on the current Mac emulator. Compared with the historical 0.74-second result, the measured speed ratio is 0.74 / 0.006952 = **106.4**.

> **Result: for this CPU-bound assembler workload, the current R65 emulator runs at approximately 106 times the speed of the historical 6502 system.**

4. R65 Pascal comparison

The same optimized Sieve algorithm was then implemented in R65 Pascal. R65 Pascal emulates a 16-bit machine, so the flag array was represented as 8192 integers. The measured execution times were 0.67 s for RUNS=1 and 6.56 s for RUNS=10.

Writing the total time as startup S plus per-Sieve time T gives S + T = 0.67 and S + 10T = 6.56. Hence 9T = 5.89 and T = **0.6544 s per Sieve**. The implied startup/loading component is only about 0.016 s.

Comparing Pascal with the native assembler measurement gives 0.6544 / 0.006952 = **94.1**. Thus, for this particular tight computational workload, interpreted R65 Pascal is approximately **94 times slower** than optimized native 6502 assembler.

This factor is specific to this benchmark; it should not be treated as a universal Pascal-versus-assembler ratio. The Sieve is dominated by tight loops, indexing, arithmetic and memory access - exactly the kind of workload where an interpreter is at a disadvantage.

5. A striking historical comparison

The two independent measurements happen to meet at an interesting point:

System / implementation	One Sieve
Historical 6502, optimized assembler	~0.74 s
R65 Pascal on current Mac emulator	~0.65 s
6502 assembler on current Mac emulator	~0.00695 s

In other words, the present emulator has accelerated the virtual 6502 sufficiently that interpreted R65 Pascal now executes this benchmark in roughly the same wall-clock time as highly optimized assembler did on the historical machine.

6. Consequences for MILL / TEKMILL

The benchmark strongly supports the conclusion that computationally intensive parts of the MILL AI would have needed assembler optimization to be comfortably usable on original R65 hardware. A convenient rule of thumb from the measured CPU ratio is approximately 100:1:

Current emulator	Approx. original-speed R65
10 ms	1 s
100 ms	10 s
500 ms	50 s
1 s	100 s

This scaling applies to CPU-bound work, not to disk I/O or Tektronix drawing. A historically plausible implementation could therefore retain the overall game, user interface, board handling and higher-level strategy in Pascal while moving the innermost AI search and evaluation loops into 6502 assembler.

7. Tektronix and PDP-11 context

The Tektronix side appears considerably less problematic than the AI computation. Experiments with TEKMILL showed little visible difference between 100 kbaud and 19,200 baud because terminal

drawing/deflection and erase time become limiting factors. At 9600 baud the interface remains practical; at 2400 baud redraws become slow, but are still representative of the waits accepted on historical modem-connected graphics terminals.

The PDP-11 is relevant primarily as historical context. Substantial programming was done in C on a PDP-11 with a Tektronix 4010, as well as in interpreted Pascal on the PDP-11, although that software has been lost. An original-speed PDP-11 emulator is still available. Porting the present C R65 emulator to that environment would be possible but would require adaptation to the older C grammar and is not necessary merely to establish the R65 performance picture.

8. Conclusions

Measured CPU speed ratio: current Mac R65 emulator is approximately 106x the historical 6502 speed for the optimized assembler Sieve.
Measured interpreter cost: R65 Pascal is approximately 94x slower than optimized 6502 assembler for this Sieve workload.

These measurements put the historical feasibility discussion of TEKMILL on firmer ground. The Tektronix interface itself is quite plausible at historically realistic communication rates. The main limitation on an original R65 would have been the CPU cost of sophisticated game-tree searching in interpreted Pascal. Selective assembler implementation of the critical AI loops would therefore have been the natural engineering solution.

Benchmark measurements recorded 11 October 2026. Times are wall-clock measurements and should be understood as practical engineering benchmarks rather than cycle-exact processor characterization.